

Klard — Anfrage-/Angebots- & Abo-Paket · Replit-Integration

Komplettes Paket zur Erweiterung deiner bestehenden Klard-Plattform um zwei neue Umsatzströme: **Pay-per-Lead** (Anfrage/Angebot) und **Abo-Tiers** (MRR).

Inhalt

```
sql/
  klard-anfragen-modul.sql → Anfragen, Angebote, Wallet, Lead-Gebühren, Reviews
  klard-abo-modul.sql      → Abo-Pläne, Subscriptions, Lead-Limits, Entitlement
  00-run-all.sql          → Reihenfolge-Hinweis
src/
  middleware/entitlements.js → Tier-Gating, Lead-Limit, Tier-Rabatt
  routes/requests.js        → Anfrage stellen, Angebot senden, annehmen, Lead-Re
  routes/subscriptions.js    → Abo starten/kündigen + Webhook-Handler
  routes/walletWebhook.js    → Guthaben-Aufladung verbuchen
emails/
  new_request_provider.hbs   → Anbieter: neue Anfrage
  offer_received.hbs         → Kunde: neues Angebot
  subscription_activated.hbs → Anbieter: Abo aktiviert
```

Schritt 1 — Datenbank migrieren

```
psql $DATABASE_URL -f sql/klard-anfragen-modul.sql
psql $DATABASE_URL -f sql/klard-abo-modul.sql
```

Beide bauen per Foreign Key auf deinen vorhandenen Tabellen auf (`branches` , `service_categories` , `service_templates` , `providers` , `customers`).

Schritt 2 — Stripe-Preise anlegen

Im Stripe-Dashboard zwei wiederkehrende Preise (monatlich) anlegen: Klard Plus (29 €) und Klard Pro (79 €). Dann die Price-IDs eintragen:

```
UPDATE subscription_plans SET stripe_price_id = 'price_XXXX' WHERE tier = 'plus';
UPDATE subscription_plans SET stripe_price_id = 'price_YYYY' WHERE tier = 'pro';
```

Schritt 3 — Routes registrieren

In deinem Express-Server (z. B. `src/index.js`):

```
import requestsRouter from "../routes/requests.js";
import subscriptionsRouter from "../routes/subscriptions.js";

app.use("/api", requestsRouter);
app.use("/api", subscriptionsRouter);
```

Schritt 4 — Webhook-Handler einhängen

Du hast bereits einen Stripe-Webhook. Ergänze nach der Signaturprüfung die neuen Handler. **Wichtig:** der Webhook braucht den Raw-Body (`express.raw`).

```
import { handleSubscriptionWebhook } from "../routes/subscriptions.js";
import { handleWalletWebhook } from "../routes/walletWebhook.js";

app.post("/webhook/stripe",
  express.raw({ type: "application/json" }),
  async (req, res) => {
    let event;
    try {
      event = stripe.webhooks.constructEvent(
        req.body, req.headers["stripe-signature"], process.env.STRIPE_WEBHOOK_SEC
      )
    } catch (err) {
      return res.status(400).send(`Webhook Error: ${err.message}`);
    }
    // deine bestehenden Handler (booking checkout etc.) ...
    await handleSubscriptionWebhook(event);
    await handleWalletWebhook(event);
    res.json({ received: true });
  });
```

Im Stripe-Dashboard diese Events abonnieren: `checkout.session.completed`,
`customer.subscription.updated`, `customer.subscription.deleted`,
`invoice.payment_failed`.

Schritt 5 — E-Mail-Templates registrieren

Lege die drei `.hbs`-Dateien zu deinen bestehenden 12. Falls deine `sendEmail()` die

Templates per Name lädt, sind keine Code-Änderungen nötig. Übergib `app_url` global im Template-Kontext (alle drei nutzen `{{app_url}}`).

Schritt 6 — Frontend (minimal)

- **Kunde:** Anfrage-Formular → `POST /api/requests` (mit `consent_data_share: true`).
- **Kunde:** Angebote vergleichen → `GET Angebote`, `POST /api/offers/:id/accept` .
- **Anbieter-Dashboard:** Tab „Anfragen“ → Liste, „Angebot senden“ → `POST /api/requests/:id/offers` .
- **Anbieter-Dashboard:** Tab „Tarif“ → `GET /api/plans` , `POST /api/subscriptions/checkout` .
- **Wallet:** „Guthaben aufladen“ → `POST /api/wallet/topup` .

Umsatzlogik auf einen Blick

Strom	Wann	Wo im Code
Buchungsprovision 7%	Slot-Buchung (bestehend)	dein vorhandener Flow
Pay-per-Lead	Anbieter sendet Angebot	<code>requests.js</code> → <code>calcLeadPrice</code>
Abo (MRR)	monatlich	<code>subscriptions.js</code>
Lead-Garantie (Refund)	toter Lead	<code>requests.js</code> → <code>/leads/:id/refund</code>

Cashflow-Hebel: Wallet (Prepaid) bringt Geld vor dem Abschluss; Abos bringen planbaren MRR; Tier-Rabatt auf Leads macht das Abo für Vielnutzer attraktiv.

DSGVO-Hinweise (DE)

- `consent_data_share` ist Pflicht, bevor eine Anfrage angelegt wird (Route lehnt sonst ab).
- `consent_timestamp` wird gespeichert (Nachweis).
- Kundendaten werden erst nach Lead-Kauf an den Anbieter sichtbar — im Frontend vor dem Angebot nur anonymisiert anzeigen (PLZ/Stadt, nicht Name/Telefon).

- Lead-Preise transparent ausweisen (Wettbewerbsrecht).